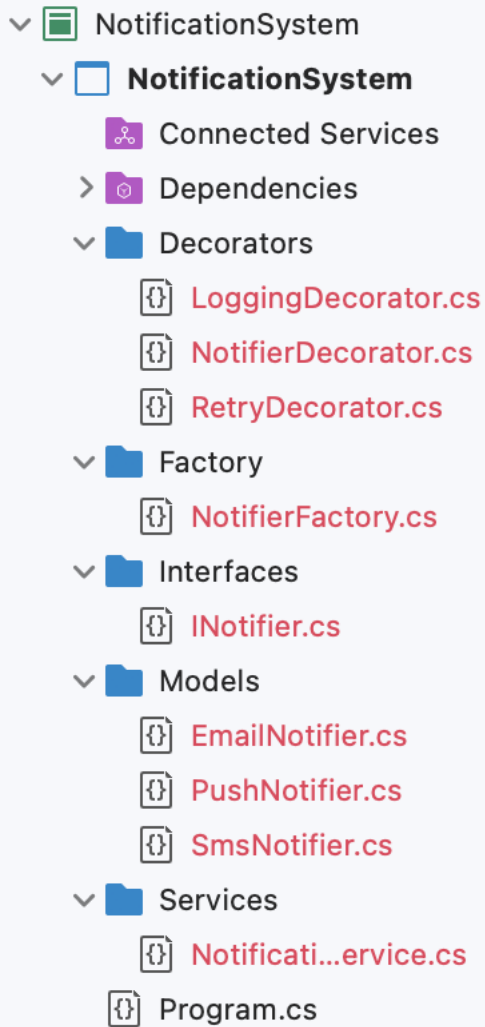




LoggingDecorator.cs

```
using System;
```

```
using NotificationSystem.Interfaces;
```

```
namespace NotificationSystem.Decorators
```

```
{
```

```
    public class LoggingDecorator : NotifierDecorator
```

```
    {
```

```
        public LoggingDecorator(INotifier notifier) : base(notifier) { }
```

```
        public override void Send(string message)
```

```
    {
```

```

        Console.WriteLine($"[LOG] Start sending message at: {DateTime.Now}");

        base.Send(message);

        Console.WriteLine("[LOG] Message process completed");
    }
}
}

```

NotifierDecorator.cs

```

using NotificationSystem.Interfaces;

namespace NotificationSystem.Decorators
{
    public abstract class NotifierDecorator : INotifier
    {
        protected readonly INotifier _wrappedNotifier;

        protected NotifierDecorator(INotifier notifier)
        {
            _wrappedNotifier = notifier;
        }

        public virtual void Send(string message)
        {
            _wrappedNotifier.Send(message);
        }
    }
}

```

RetryDecorator.cs

```

using System;

using NotificationSystem.Interfaces;

namespace NotificationSystem.Decorators
{
    public class RetryDecorator : NotifierDecorator
    {

```

```

{
    public RetryDecorator(INotifier notifier) : base(notifier) { }

    public override void Send(string message)
    {
        try
        {
            base.Send(message);
        }
        catch (Exception)
        {
            Console.WriteLine("[RETRY] Error detected. Retrying to send message...");
            base.Send(message);
        }
    }
}

```

NotifierFactory.cs

```

using System;

using NotificationSystem.Interfaces;
using NotificationSystem.Models;

namespace NotificationSystem.Factory
{
    public static class NotifierFactory
    {
        public static INotifier CreateNotifier(string type)
        {
            return type.ToLower() switch
            {
                "email" => new EmailNotifier(),
                "sms" => new SmsNotifier(),
                "push" => new PushNotifier(),
                _ => throw new ArgumentException("Invalid notification type")
            }
        }
    }
}

```

```
        };  
    }  
}  
}
```

INotifier.cs

```
namespace NotificationSystem.Interfaces  
{  
    public interface INotifier  
    {  
        void Send(string message);  
    }  
}
```

EmailNotifier.cs

```
using System;  
using NotificationSystem.Interfaces;  
  
namespace NotificationSystem.Models  
{  
    public class EmailNotifier : INotifier  
    {  
        public void Send(string message)  
        {  
            Console.WriteLine($"[Email] Sending message: {message}");  
        }  
    }  
}
```

PushNotifier.cs

```
using System;  
using NotificationSystem.Interfaces;  
  
namespace NotificationSystem.Models  
{
```

```

public class PushNotifier : INotifier
{
    public void Send(string message)
    {
        Console.WriteLine($"[Push] Sending notification: {message}");
    }
}

```

SmsNotifier.cs

```

using System;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Models
{
    public class SmsNotifier : INotifier
    {
        public void Send(string message)
        {
            Console.WriteLine($"[SMS] Sending message: {message}");
        }
    }
}

```

NotificationService.cs

```

using System;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Services
{
    public class NotificationService
    {
        private INotifier? _notifier;

        public void SetNotifier(INotifier notifier)

```

```

{
    _notifier = notifier;
}

public void SendNotification(string message)
{
    if (_notifier == null)
    {
        Console.WriteLine("[Error] Please set a notification method first");
        return;
    }
    _notifier.Send(message);
}
}
}

```

Program.cs

```

using System;

using NotificationSystem.Interfaces;
using NotificationSystem.Factory;
using NotificationSystem.Decorators;
using NotificationSystem.Services;

namespace NotificationSystem
{
    class Program
    {
        static void Main(string[] args)
        {
            // 1. Khởi tạo Service (Context của Strategy Pattern)
            NotificationService notificationService = new NotificationService();

            // 2. Sử dụng Factory để tạo Email Notifier và bọc Decorator
            INotifier emailNotifier = NotifierFactory.CreateNotifier("email");
            emailNotifier = new LoggingDecorator(emailNotifier);
        }
    }
}

```

```
emailNotifier = new RetryDecorator(emailNotifier);

// 3. Gán chiến lược (Strategy) và thực thi
Console.WriteLine("--- Scenario 1: Email with Log & Retry ---");
notificationService.SetNotifier(emailNotifier);
notificationService.SendNotification("Your ticket booking is successful");

Console.WriteLine("\n--- Scenario 2: Quick SMS without Decorators ---");
INotifier smsNotifier = NotifierFactory.CreateNotifier("sms");
notificationService.SetNotifier(smsNotifier);
notificationService.SendNotification("OTP: 123456");

Console.WriteLine("\nPress any key to exit");
Console.ReadKey();
}
}
}
```